

Please expand the documentation in the following code, maintaining and replicating the style of the comments already present.

```
def find_files(directory, extension):  
    """  
    This function is responsible for traversing  
    the filesystem rooted at the  
    provided `directory` argument. It will  
    walk every subdirectory it  
    encounters and, for each file it finds, it  
    will check whether the  
    filename ends with the string supplied in  
    `extension`. All matching  
    paths are accumulated into a list which is  
    ultimately returned to the  
    caller. The function does not follow  
    symbolic links and does not raise  
    an exception if the directory does not  
    exist - it simply returns an  
    empty list in that scenario.  
    """  
    matches = []  
    for root, dirs, files in  
os.walk(directory):  
        for fname in files:  
            if fname.endswith(extension):  
  
    matches.append(os.path.join(root, fname))  
    return matches
```

Here's an expanded version that preserves the existing tone and style—descriptive, implementation-focused, and written as a narrative explanation of the function's behavior:

 Python



 Run

```
def find_files(directory, extension):  
    """  
    This function is responsible for traversing the filesystem rooted at the  
    provided `directory` argument. It will walk every subdirectory it
```

encounters and, for each file it finds, it will check whether the filename ends with the string supplied in ``extension``. All matching paths are accumulated into a list which is ultimately returned to the caller. The function does not follow symbolic links and does not raise an exception if the directory does not exist - it simply returns an empty list in that scenario.

The returned values are absolute or relative paths depending on the form of the ``directory`` argument that was supplied. Paths are constructed by joining the directory currently being visited with the matching filename, ensuring that the returned entries can be used directly in subsequent filesystem operations.

The ``extension`` argument is treated as a simple string suffix rather than as a normalized file extension. As a result, matching is performed using a straightforward string comparison via ``str.endswith()``. This means that callers are responsible for supplying any leading period (for example, `".txt"`) if such behavior is desired, and matching is performed exactly as specified, including case sensitivity.

Files are returned in the order produced by the underlying directory traversal mechanism. No additional sorting, deduplication, or filtering is performed. Consequently, the ordering of results may vary across operating systems, filesystems, or Python versions.

The function performs a complete traversal of the directory tree before returning. For very large directory hierarchies, memory usage grows proportionally with the number of matching files because all results are stored in memory simultaneously prior to being returned to the caller.

"""

```
matches = []
for root, dirs, files in os.walk(directory):
    for fname in files:
        if fname.endswith(extension):
            matches.append(os.path.join(root, fname))
return matches
```

